

JUIC = Java UI Components

React like Java framework

Dom

Components

Tailwind integration

React architecture

Server-side React like engine

Project?

Nice—now this connects back to your earlier idea 👍

If you want to **build a React-like framework using Java**, let's turn it into a **proper project** (something you can even show in interviews).

🚀 Project: “Mini React in Java”



Goal

Build a **component-based UI framework** in Java that:

- Has **components**
- Supports **state**
- Renders **HTML**
- (Optional) Does basic **diffing** like **Virtual DOM**



What you'll learn

- How React works internally
- System design thinking
- Framework architecture (big plus for

interviews)



Project structure

```
mini-react-java/ | |—— Component.java
|—— State.java |—— Renderer.java |——
App.java |—— components/ |——
Button.java
```



Step 1: Create Component system

```
public interface Component { String
render(); }
```



Step 2: Create a simple component

```
public class Button implements Component
{ private String text; public Button(String
text) { this.text = text; } @Override public
String render() { return "<button>" + text +
"</button>"; } }
```



Step 3: Create Renderer

```
public class Renderer { public static void
render(Component component) {
```

```
System.out.println(component.render()); } }
```



Step 4: Main App

```
public class App { public static void  
main(String[] args) { Component btn = new  
Button("Click Me"); Renderer.render(btn); } }
```



Output:

```
<button>Click Me</button>
```



Step 5: Add State (React-like)

```
public class Counter implements Component  
{ private int count = 0; public void  
increment() { count++; } @Override public  
String render() { return "<h1>Count: " +  
count + "</h1>"; } }
```



Step 6: Simulate re-render

```
Counter counter = new Counter();  
Renderer.render(counter);  
counter.increment();  
Renderer.render(counter);
```



Step 7 (Advanced): Virtual DOM idea

Instead of rendering full HTML every time:

- Store old UI
- Compare with new UI
- Update only changes

👉 This is how React becomes fast

💡 Upgrade ideas (very important)

You can make this project 🔥 by adding:

✅ 1. Props support

`new Button("Submit")`

✅ 2. Nested components

`<div> <Button /> <Text /> </div>`

✅ 3. Event simulation

- onClick logic (console-based)

✅ 4. Convert to real web app

Use:

- Spring Boot

to serve HTML

🎯 Final outcome

You'll have:

- 👉 Your own **Java UI framework**
- 👉 Deep understanding of React
- 👉 Strong portfolio project

💬 **Real talk**

This won't replace React—but:

- 👉 It will make you **way better than average devs**

👉 **Next step**

If you want, I'll guide you like a real dev mentor:

Just say: “**next step**” and we'll level it up 🚀

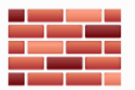
Html Elements?

Ahh got it—you want each section to restart numbering from 1 👍

That makes much more sense.



HTML Elements (Section-wise numbering)



Root & Metadata

- `<html>`
- `<head>`
- `<title>`
- `<base>`
- `<link>`
- `<meta>`
- `<style>`
- `<script>`
- `<noscript>`



Layout / Sectioning

- `<body>`
- `<header>`
- `<footer>`

- <main>
- <section>
- <article>
- <nav>
- <aside>
- <div>



Headings & Text

- <h1>
- <h2>
- <h3>
- <h4>
- <h5>
- <h6>
- <p>
- <hr>
-

- <pre>
- <blockquote>



Inline Text

- <a>
-

- `<strong>`
- `<b>`
- `<em>`
- `<i>`
- `<u>`
- `<small>`
- `<mark>`
- `<del>`
- `<ins>`
- `<sub>`
- `<sup>`
- `<code>`
- `<kbd>`
- `<samp>`
- `<var>`
- `<cite>`
- `<q>`
- `<abbr>`
- `<data>`
- `<time>`
- `<wbr>`



Lists

- `<ul>`
- `<ol>`
- `<li>`
- `<dl>`
- `<dt>`
- `<dd>`



Media

- `<img>`
- `<audio>`
- `<video>`
- `<source>`
- `<track>`
- `<map>`
- `<area>`
- `<picture>`
- `<figure>`
- `<figcaption>`
- `<iframe>`
- `<embed>`
- `<object>`
- `<param>`



Forms

- `<form>`
- `<input>`
- `<textarea>`
- `<button>`
- `<select>`
- `<option>`
- `<optgroup>`
- `<label>`
- `<fieldset>`
- `<legend>`
- `<datalist>`
- `<output>`



Tables

- `<table>`
- `<caption>`
- `<thead>`
- `<tbody>`
- `<tfoot>`
- `<tr>`
- `<th>`

- `<td>`
- `<colgroup>`
- `<col>`



Interactive

- `<details>`
- `<summary>`
- `<dialog>`



Misc

- `<progress>`
- `<meter>`
- `<canvas>`

Now this format is clean and easier to study



If you want next:  

Architecture of React?

Good — now let's look at **React architecture**, but in a way that actually helps you build your JUIC framework.

I'll map everything to **system design + DS concepts**, not just theory.

High-Level Architecture of React

Think of React as 3 main layers:

Components → Virtual DOM →

Reconciliation → Real DOM

1. Component Layer (Entry Point)

👉 Everything starts with **components**
`function App() { return <h1>Hello</h1>; }`

In DS terms:

- Component = Function that returns a Tree (VNode tree)

2. Virtual DOM (VDOM)

👉 React converts components → **Virtual DOM Tree**

Example:

VNode Tree: h1 / "Hello"

In DS terms:

- N-ary Tree
- Stored in memory
- Lightweight representation of UI



3. Reconciliation (Diffing Engine)

This is the **core algorithm**

👉 React compares:

- Old VDOM (previous state)
- New VDOM (after change)

In DS terms:

- Tree comparison using DFS
- Generates patch operations

Key Optimizations React uses:

1. Same-level comparison

👉 Doesn't compare across levels (reduces

complexity)

2. Keys (Hashing)

```
<li key="1">Item</li>
```

👉 Like:

- `HashMap<key, node>`
- Fast lookup instead of index-based

4. Fiber Architecture (Advanced Core)

This is where React becomes powerful.

👉 React internally uses **Fiber Tree**

What is Fiber?

- A linked tree structure
- Each node = unit of work

Fiber Node: - parent - child - sibling

In DS terms:

👉 Tree + Linked List hybrid

Why Fiber exists?

To enable:

- Interruptible rendering
- Scheduling

- Smooth UI (no blocking)



5. Scheduler (Task Manager)

React doesn't update everything at once.



It schedules work like:

- High priority → user input
- Low priority → background updates

In DS terms:

- Priority Queue
- Task scheduling system



6. Renderer (React DOM)



Final step: Convert VDOM → Real DOM

Example:

```
<h1>Hello</h1>
```

In DS terms:

- Tree → Actual system output



Full Flow (Step-by-Step)

1. State changes 2. Component re-runs 3. New VDOM created 4. Diff with old VDOM (Reconciliation) 5. Generate patches 6. Apply

updates to Real DOM



Architecture in One View

[Component] ↓ [Virtual DOM (Tree)] ↓ [Reconciliation (Diff - DFS)] ↓ [Fiber (Work Units + Scheduling)] ↓ [Renderer (DOM Updates)]



Mapping to Your JUIC Framework

ReactYour JUICComponentJava
classVDOMVNodeReconciliationdiff()Fiber(skip for now)RendererHTML generator



Reality Check

You should NOT try to build:

- Fiber
- Scheduler



Too complex for now



What you SHOULD build

Focus on:

- Component system 
- VNode tree 

- Diffing (DFS) 
- Renderer 

👉 That's enough to impress in interviews

Interview-level explanation

If asked:

“Explain React architecture”

You say:

“React uses a component-based architecture where UI is represented as a Virtual DOM tree (an N-ary tree). On state updates, it creates a new tree and performs reconciliation using a DFS-based diffing algorithm. It then applies minimal updates to the real DOM. Internally, React uses Fiber for scheduling and prioritizing rendering tasks.”

If you want next:

- I can
- Or

Just say 👉 “*JUIC architecture*”

Diffing concept in ds terms?

Good question — this is exactly how you should think if you want to build JUIC properly.

Let's strip away “frontend talk” and explain everything in **Data Structures (DS)** terms.



1. VNode = Tree Node

Your **VNode** is just a node in a **Tree** data structure

```
class VNode { String tag; // node value
String text; // data Map<String, String>
props; // metadata List<VNode> children; //
edges to child nodes }
```

👉 In DS language:

- **VNode = Tree Node**
- **children = Adjacency list (N-ary tree)**
- **Whole UI = Tree**



2. UI = N-ary Tree

Unlike binary trees, here:

👉 Each node can have **multiple children**

Example:

```
<div> <h1>Hello</h1> <p>Text</p> </div>
```

Tree:

div / \ h1 p

👉 This is an N-ary Tree



3. Diffing = Tree

Comparison Problem

Now the real part:

👉 Diffing = Compare two trees

- Old Tree (T1)
- New Tree (T2)

Goal:

Find minimum changes to convert $T1 \rightarrow T2$



In DS terms:

This is similar to:

- Tree traversal
- Tree matching
- Structural comparison



4. Algorithm (Recursive Tree Traversal)

You are basically doing:

👉 DFS (Depth First Search)

Patch diff(VNode oldNode, VNode newNode)

Step-by-step (DS logic)

Case 1: Node doesn't exist

if (oldNode == null || newNode == null)

👉 Insert/Delete operation

Case 2: Node value different

if (!oldNode.tag.equals(newNode.tag))

👉 Replace node

(Like replacing a tree node)

Case 3: Data changed

if (!oldNode.text.equals(newNode.text))

👉 Update node value

Case 4: Same node → go deeper

for each child: diff(oldChild, newChild)

👉 Recursive DFS



5. Time Complexity

Naive tree comparison:

👉 $O(n)$ where n = number of nodes

Because:

- You visit each node once

⚠ Important Note (Real-world)

General tree edit distance is expensive: 🙌
 $\sim O(n^3)$

But React optimizes it using assumptions:

- Same level nodes compared
- Keys for identification

🙌 So it becomes $O(n)$

🔑 6. Keys = Hashing Optimization

When you use:

String key;

🙌 This is like:

- HashMap lookup
- Avoids index-based matching

Instead of:

child[0] vs child[0]

You do:

map[key] $\rightarrow$ node

🙌 Faster matching



7. Patch = Set of Operations

In DS terms:



You are generating operations list

Like:

- INSERT node
- DELETE node
- UPDATE node
- REPLACE subtree

This is similar to:

- Edit operations
- Transformation steps



Final Mapping (DS vs JUIC)

JUIC ConceptData Structure

ConceptVNodeTree NodeUIN-ary

TreechildrenAdjacency ListDiffingTree

ComparisonRecursionDFS

TraversalkeyHashing / MapPatchEdit

Operations



What you should

understand deeply

If interviewer asks:

👉 “How does your framework work?”

You say:

“I represent UI as an N-ary tree. I perform a DFS-based comparison between old and new trees, generating patch operations like update, replace, and delete. I use keys for efficient node matching similar to hashmap lookups.”

That answer alone puts you ahead of most candidates.

If you want next level:

- I can show **optimized diff using HashMap (keys)**
- Or **how to apply patches efficiently**

Just say 👉 “*optimize diff*” or “*apply patches DS way*”

How can I build a React like framework in java?

You *can* build a React-like framework in Java — just don't aim to replace React in the browser. Build a **server-side / learning framework (JUIC)** that uses the same ideas: components → virtual DOM → diff → render. Here's a **clean, practical blueprint** you can actually finish.



1. Decide your target (important)

Pick one:



Recommended (for you)

Server-side renderer (SSR-like)

- Input: Java components
- Output: HTML string
- Runs with Spring Boot



This is realistic, interview-friendly, and useful.



2. Core Architecture (JUIC)

Component → VNode Tree → Diff → Patch
→ Renderer → HTML

You'll build this in layers.

3. Step-by-step Implementation

◆ Step 1: VNode (Tree Node)

```
class VNode { String tag; String text;  
Map<String, String> props = new  
HashMap<>(); List<VNode> children = new  
ArrayList<>(); String key; public  
VNode(String tag) { this.tag = tag; } public  
VNode text(String text) { this.text = text;  
return this; } public VNode child(VNode  
child) { this.children.add(child); return this; }  
public VNode prop(String k, String v) {  
this.props.put(k, v); return this; } }
```

👉 DS view: N-ary tree node

◆ Step 2: Component System

```
abstract class Component { public abstract  
VNode render(); }
```

Example:

```
class App extends Component { public  
  VNode render() { return new VNode("div")  
    .child(new VNode("h1").text("Hello JUIC")); }  
}
```

◆ Step 3: Renderer (VNode → HTML)

```
class Renderer { public static String  
  render(VNode node) { if (node == null) return  
    ""; StringBuilder html = new StringBuilder();  
    html.append("<").append(node.tag); // props  
    for (var entry : node.props.entrySet()) {  
      html.append(" ").append(entry.getKey())  
        .append("=\\"").append(entry.getValue())  
        .append("\\"); } html.append(">"); if  
    (node.text != null) { html.append(node.text);  
    } for (VNode child : node.children) {  
      html.append(render(child)); }  
    html.append("</").append(node.tag).append(  
    ">"); return html.toString(); } }
```

◆ Step 4: Diffing (Tree

Comparison)

```
class Patch { String type; // REPLACE, TEXT, NONE
VNode node; public Patch(String type, VNode node) { this.type = type; this.node = node; } }
class Diff { public static Patch diff(VNode oldNode, VNode newNode) { if (oldNode == null) return new Patch("REPLACE", newNode); if (!oldNode.tag.equals(newNode.tag)) { return new Patch("REPLACE", newNode); } if (!Objects.equals(oldNode.text, newNode.text)) { return new Patch("TEXT", newNode); } return new Patch("NONE", null); } }
```

◆ Step 5: Engine (Core Runner)

```
class JUIC { private VNode oldTree; public String render(Component component) { VNode newTree = component.render(); Patch patch = Diff.diff(oldTree, newTree); oldTree = newTree; // For now: full render return Renderer.render(newTree); } }
```



4. Connect with Spring

Boot (real-world touch)

Create controller:

```
@RestController public class PageController
{ @GetMapping("/") public String home() {
  JUIC juic = new JUIC(); return juic.render(new
  App()); } }
```

👉 Open browser → see HTML

🔥 5. Folder Structure (clean design)

```
juic/ |—— core/ | |—— VNode.java | |——
Component.java | |—— Renderer.java | |——
Diff.java | |—— JUIC.java |—— app/ | |——
App.java |—— controller/ |——
PageController.java
```

🚀 6. What to build next (this is where it becomes impressive)

After basic version works:

Level 2

- Props rendering ✓
- Children diffing ✓
- Patch apply (instead of full render)

Level 3

- Add key support (HashMap diff)
- State system: setState(...)
- Re-render trigger

Level 4 (advanced)

- Template syntax (like JSX alternative)
- CLI tool
- Routing system

Reality Check

- Browsers don't run Java → so no direct DOM control
- This is closer to:
- Server-side rendering engine
- Template engine (but smarter)



Why this helps your career

You can say:

"I built a React-like framework in Java implementing Virtual DOM, diffing, and

server-side rendering”

That signals:

- Strong DS knowledge
- System design thinking
- Real engineering mindset



What you should do now

Don't try to build everything at once.



Start with:

- VNode
- Renderer
- Simple Component

Get output working in browser.

If you want, I can next:

- Or
- Or

Just say 

Where will the html elements be?

Good — this is the exact question that separates “idea” vs “real system” thinking.

Short answer:

👉 In your Java-based JUI framework, HTML elements don't physically exist until you render them

👉 They are first Virtual Nodes (in memory) → then converted to HTML string → then shown in browser

🧠 **3 Levels where your “HTML elements” exist**

🟡 **1. Inside Java (Virtual DOM)**

Here, elements are just objects in memory

```
new VNode("h1").text("Hello")
```

👉 This is NOT real HTML

👉 It's just a tree data structure

🟢 **2. After Rendering (HTML String)**

Your Renderer converts VNode → HTML:

<h1>Hello</h1>

👉 Now it becomes real HTML code (string)

🔵 3. In Browser (Real DOM)

When you return this from Spring Boot:

```
@GetMapping("/") public String home() {  
    return "<h1>Hello</h1>";  
}
```

👉 Browser parses it → creates Real DOM

🔄 Full Flow (very important)

Java Component ↓ VNode Tree (memory) ↓
Renderer ↓ HTML String ↓ HTTP Response ↓
Browser ↓ Real DOM (actual elements)

📌 So where are the elements actually?

StageWhere elements existBefore
renderJava objects (VNode)After
renderHTML stringIn browserReal DOM
elements

⚠️ Important Reality

Your framework (JUIC):

👉 ❌ Does NOT control browser DOM

directly

👉 ✅ Generates HTML (like server-side rendering)

🤔 **Then how is this different from React?**

React:

- Runs in browser (JavaScript)
- Directly updates DOM

Your JUIC:

- Runs on server (Java)
- Sends HTML to browser

👉 So yours is closer to:

- JSP
- Thymeleaf
- SSR engines

💡 **If you WANT real DOM control (advanced idea)**

You'd need:

Option 1:

- Send JSON (VNode) to frontend
- Use JavaScript to render

Option 2:

- Build your own JS runtime (very complex)



Smart move (recommended)

Keep JUIC as:

👉 **Server-side React-like engine**

Later you can extend:

- Add API endpoints
- Send data dynamically
- Re-render pages



Interview-level answer

If someone asks:

“Where are your HTML elements in your framework?”

You say:

“They exist as Virtual Nodes in Java memory, then are rendered into HTML strings on the server, and finally parsed into real DOM

elements by the browser.”

If you want next:

- I can show
- Or

Just say 👉 *“partial update”* or *“make it dynamic”*

Does the market need anything like this?

Short answer:

👉 The market does NOT need a React-like framework in Java (for frontend)

👉 But it DOES value the knowledge and niche use-cases behind it

Now let's break this down honestly.



1. Market Reality (very important)

Today's frontend world is dominated by:

- React
- Angular
- Vue
- Next.js

👉 React is still the most in-demand frontend tech in 2026

Why?

Because:

- Browsers run **JavaScript**, not Java
- Huge ecosystem (libraries, tools, jobs)
- Companies already invested in it

2. Java-based frontend frameworks (truth)

Java tried this before:

- Applets  (dead)
- JSP / JSF  (declining)
- Server-side rendering  (reduced popularity)



Industry shifted toward:

Client-side apps (SPA) using JavaScript frameworks



3. So is your idea useless?

No — but your positioning matters

Let's be clear:

 If your goal is:

“Replace React with Java”

→ This will fail

→ Market doesn't want it



If your goal is:

“Build a React-like system in Java for learning / niche use”

→ This is VERY valuable



4. Where your idea actually fits (real opportunities)



1. Learning / Portfolio (BEST for you)

This is HUGE:

You demonstrate:

- Data structures (tree, diffing)
- System design
- Framework thinking



This helps you get a developer job



2. Internal enterprise tools

Some companies still prefer:

- Java everywhere
- Server-side rendering

Example:

- Admin dashboards
- Internal tools



Java UI frameworks still exist here (but

niche)

3. Modern trend: Server-side UI (important insight)

Here's the interesting part:

👉 Even React is moving BACK toward server rendering

- React Server Components reduce client JS
- Faster load times (~70% improvements reported)

👉 This means:

Your idea (server-driven UI) is conceptually relevant

5. What the market **ACTUALLY** needs

Not “React in Java”

But:

✓ **Backend + Frontend integration**

- Java (Spring Boot) + React

✓ **Full-stack developers**

- API + UI
- ✓ **Performance-focused rendering**
 - SSR / hybrid apps

💣 **Brutally honest conclusion**

✗ **Market does NOT need:**

- New Java frontend framework
- React replacement in Java

✓ **Market DOES value:**

- Developers who understand React architecture deeply
- Engineers who can build systems like frameworks
- People who can explain Virtual DOM, diffing, etc.

🎯 **Best strategy for YOU**

Do this smartly:

1. Build JUIC (your framework)

👉 For learning + resume

2. Learn real stack:

- Spring Boot

- React

3. Combine both in interviews:

Say:

“I built my own React-like framework in Java to understand Virtual DOM and diffing”

👉 That's powerful



Final mindset shift

Don't build JUIC to “sell it”

👉 Build it to upgrade yourself

If you want, I can help you turn JUIC into:

- A resume project that gets interviews
- Or a GitHub project that stands out

Just say 👍